

Computable Logic Through Dynamic Measureless Connections

John Phantom

johnphantom@hotmail.com

ABSTRACT

A measureless, operator–interactive finite–state logic where computation is encoded purely in connection topology. – offering a unique topological implementation of a finite-state machine.

This paper presents a novel computational framework that implements logical operations through dynamic connection topology in constrained scripting environments, independent of numerical measurement or evaluation. I demonstrate that the Quakeworld engine's command architecture enables the construction of fundamental logical and arithmetic operations using only connection manipulation, requiring external operator intervention rather than automated execution.

INTRODUCTION

Logic Geometry demonstrates that logical behavior can arise from connection topology alone, without measurement, without value-bearing symbolic state representations, or explicit state variables, fundamentally challenging the foundational assumption that computation requires a distinction between structure (the system's form) and content (the values or symbols the system processes). In classical logic and computation theory, information is encoded in measurable states—Boolean values, numerical registers, symbolic tokens—and the relationship between states. Logic Geometry inverts this: the connection configuration itself is simultaneously the structure, the content, and the state. There is no distinction between "the system" and "what the system represents"—the shape of connections is the information; the logic is the topology.

Using only three primitive operations—alias (command definition), bind (input mapping), and echo (output)—available within the Quakeworld scripting environment, I demonstrate that conditionals, loops, and bounded arithmetic can be implemented without explicit state variables or numerical evaluation. Though non-Turing complete and operator-dependent, this framework reveals that logical behavior can arise from connectivity alone, with implications for understanding computation independent of measurement and representation.

In the Quakeworld implementation, command names are required by the host engine as addressable handles for defining and rebinding connections. These names are implementation artifacts of the substrate, not value-bearing symbols within the Logic Geometry model itself. The computation does not inspect, compare, parse, or branch on alias names; it evolves only through the current pattern of connectivity induced by rebinding. Accordingly, symbol use in this implementation is infrastructural rather than semantic.

Comparison to Standard Models

Comparison to Membrane Computing:

Logic Geometry shares membrane computing's emphasis on structural topology as the computational substrate rather than symbolic manipulation, but differs fundamentally in its primitives and abstraction level. Membrane computing (P systems) encodes computation through hierarchical membrane

structures, object multisets, and rewriting rules that govern how objects move between membranes or how membranes divide and dissolve—the membrane configuration and its evolution define the computation. Logic Geometry, by contrast, operates at a more minimal level: computation arises solely from dynamic connection redefinition (via alias) without objects, tokens, or hierarchical containers. Where membrane systems use membranes as boundaries that regulate object flow, Logic Geometry uses connection topology itself as both structure and information, with no distinction between "container" and "content"—the shape of the command graph is the state. Both frameworks demonstrate that computation can emerge from relational structure rather than value measurement, but membrane computing retains symbolic objects and rewriting rules, whereas Logic Geometry does not require value bearing symbols, making it a more radically measureless and connection-pure model. [1, 2, 3]

Comparison to Chemical Reaction Networks:

Chemical Reaction Networks (CRNs) model computation through molecular species concentrations and reaction rules that transform one set of molecular counts into another, with the network topology (which species react to produce which products) determining computational behavior. Like Logic Geometry, CRNs demonstrate that computation can arise from structural connectivity—the graph of possible reactions—rather than explicit symbolic manipulation. However, CRNs fundamentally rely on quantitative measurements: molecular concentrations, reaction rates, and stoichiometric coefficients are central to their computational semantics, whether in deterministic (mass-action kinetics) or stochastic (Gillespie algorithm) implementations. Logic Geometry differs by eliminating measurement entirely—there are no concentrations, no rate constants, no numerical values of any kind. Where a CRN computes by tracking how many molecules of each species exist and evolving those counts according to reaction rules, Logic Geometry computes by redefining which commands connect to which other commands, with the connection topology itself (not any measured quantity) encoding state. Both frameworks show that network structure can be computational, but CRNs are measurement-based (concentration dynamics), while Logic Geometry is purely topological (connection reconfiguration without quantities). [4, 5, 6]

Comparison to Petri Nets:

Petri nets represent computation as the movement and transformation of tokens through a directed graph of places and transitions, where token positions encode state and transition firings (governed by enabling rules) define state evolution. Like Logic Geometry, Petri nets abstract computation away from symbolic manipulation and ground it in graph structure and state transitions—the topology of places, transitions, and arcs determines what computations are possible. However, Petri nets retain explicit state representation via token counts: each place holds a discrete number of tokens, and computation proceeds by decrementing tokens from input places and incrementing tokens in output places according to transition rules. Logic Geometry eliminates this token-based state representation entirely. Where a Petri net computes by tracking token distribution across places and firing transitions when input conditions are met, Logic Geometry computes by dynamically redefining command connections without any intermediate state representation—the current connection configuration is the entire state, with no tokens, counts, or discrete objects to track. Both frameworks demonstrate that graph topology and state transitions can substrate computation, but Petri nets are token-based (discrete object movement), while Logic Geometry is purely relational (connection redefinition with no objects or measurements). [7, 8, 9]

Comparison to Graph Rewriting Systems:

Graph rewriting systems compute by applying transformation rules to graph structures, where each rule specifies a pattern to match in the current graph and a replacement subgraph to substitute—computation proceeds as a sequence of graph rewrites until a normal form or terminal state is reached. Like Logic Geometry, graph rewriting grounds computation in structural transformation rather than value evaluation, and the graph topology itself (which nodes and edges exist, how they connect) is the primary computational object. However, graph rewriting systems typically operate on explicit graph nodes and edges as discrete entities, and the rewrite rules must specify pattern matching (which nodes match the rule) and substitution (how to replace matched subgraphs)—this requires either symbolic node labels or numerical node identifiers to distinguish nodes and apply rules correctly. Logic Geometry differs by eliminating discrete entities entirely: there are no nodes or edges to label, no pattern matching on symbols, no replacement operations on subgraphs. Instead, computation arises from direct redefinition of connections (via alias commands), where the current connection configuration is simultaneously the state, the rule executor, and the transition mechanism—a single unified topology that evolves without reference to explicit symbols or intermediate representations. Both frameworks show that graph structure can be computational, but graph rewriting requires symbolic or numerical entity labels to enable pattern matching and substitution, while Logic Geometry is purely relational and unmarked—the shape and reconfiguration of connections alone suffice. [10, 11, 12]

Comparison to Lambda Calculus:

Lambda Calculus formalizes computation as the application and reduction of symbolic function expressions, where variables, abstractions, and applications are manipulated according to syntactic rewrite rules; computation is defined by beta-reduction on symbolic terms in an unbounded, abstract space of expressions. Like Logic Geometry, Lambda Calculus provides a minimal, foundational model of computation, but it remains inherently symbolic and value-centric: terms have structure independent of the reduction process, and the semantics of computation rely on substitution of variable bindings and evaluation of expression structure. Logic Geometry differs by eliminating variables, symbols, and substitution entirely—there are no expressions to rewrite, no bound variables, no environments. Where Lambda Calculus computes by rewriting expressions according to syntactic rules, Logic Geometry computes by reconfiguring a network of connections, with the current connection topology simultaneously serving as program, data, and state. Both frameworks aim at extreme minimality, but Lambda Calculus is axiomatically algebraic and symbolic, whereas Logic Geometry is purely topological and relational, suggesting a complementary foundation for computation that does not presuppose symbols, measurement, or substitution at all. [13, 14, 15]

Framework	Substrate	Basic Unit	Information Encoding	Requires Measurement?	Requires Symbols?
Membrane Computing	Hierarchical membranes	Objects/multisets	Object counts & membrane structure	Yes (implicitly)	Yes (object types)
Chemical Reaction Networks	Molecular species	Molecules	Concentration dynamics	Yes (essential)	Yes (species)
Petri Nets	Places & transitions	Tokens	Token distribution	No	Partially (labels)
Graph Rewriting	Graph nodes & edges	Subgraphs	Node/edge structure	No	Yes (node labels)

Framework	Substrate	Basic Unit	Information Encoding	Requires Measurement?	Requires Symbols?
Lambda Calculus	Abstract expression space	Lambda terms	Symbolic term structure and substitutions	No	Yes (variables, symbols)
Logic Geometry	Connection topology	Connections	Topology shape alone	No	No

1.0 Relationship to Turing Machines

Proposition 1.0 (Non-Turing Completeness)

Claim: Logic Geometry as defined is not Turing-complete.

Proof sketch:

1. Every Logic Geometry configuration has finite N (finite nodes).
2. Topology changes are operator-dependent (no autonomous infinite execution).
3. No unbounded tape or infinite workspace exists.
4. Therefore, Logic Geometry cannot simulate arbitrary Turing machines.
5. The model is at most as expressive as operator-interactive finite-state systems.

This locates Logic Geometry as a distinct computational class rather than a universal programming language.

2.0 Relationship to Finite State Machines

Proposition 2.0 (FSM Representation)

Claim: Every deterministic finite automaton (DFA) can be represented as a Logic Geometry configuration.

Proof sketch:

1. Let $M=(Q, \Sigma, \delta, q_0, F)$ be a DFA.
2. Construct configuration C_M :
 - Create node n_q for each state $q \in Q$.
 - For each q , define $\text{body}(n_q)$ to contain nodes implementing state transitions.
 - For each input symbol $\sigma \in \Sigma$, create node input_σ that redefines “current” to point to $n\delta(q,\sigma)$.
 - Accepting states trigger $\text{ECHO}(\text{"accept"})$.
3. Show that input sequence $w \in \Sigma^*$ is accepted by MM if the corresponding operator inputs to C_M produce "accept" output.
4. Conclude that DFAs embed into Logic Geometry.

Corollary 2.0

Logic Geometry is at least as expressive as regular languages.

MATERIALS AND METHODS

The Operator:

The operator is an external agent — a human — who supplies the sole input to a Logic Geometry configuration by pressing bound keys. The operator does not inspect, read, or evaluate the internal connection state of the system. They only press a key. Everything that happens as a result is determined entirely by the current connection topology, not by any judgment the operator makes about that topology. The operator's only window into the system is whatever the system chooses to echo to the console.

Logic Implementation:

The computational models developed in this study employ the Quakeworld scripting engine and its derivatives, including Counter-Strike, as the experimental platform. The primary mechanism for logic implementation utilizes the "alias" system command, which enables the definition and redefinition of command sequences. A distinguishing feature of the proposed approach is that command strings execute exclusively through invocation of previously defined alias operations, creating self-referential execution chains that propagate information solely through the topology of established connections, independent of explicit value measurement or numerical state representation.

Input/Output Interface:

Logic state externalization is facilitated through two complementary system commands that serve as the model's interface layer. The "bind" command establishes deterministic mappings between physical input events—specifically, keyboard inputs—and corresponding computational procedures, thereby enabling user-directed manipulation of system state. The "echo" command provides the dual functions of state interrogation and output communication, rendering the current logical configuration as text to the console. Together, these commands constitute the complete input-output apparatus for the measurement-free logical system, maintaining the principle that all information propagation occurs through connection topology rather than explicit numerical evaluation.

Principal Arguments and Methods

- The methodology is based on scripting within game engines (Quakeworld/Counter-Strike), utilizing only three system commands:
 - `alias`: creates or redefines command sequences ("connections"). alias format: 'alias <command name> "<commands>"
 - `bind`: links physical inputs to these connection scripts. bind format: 'bind <key> <command>'
 - `echo`: outputs or communicates current logic states or results. echo format: 'echo <string of text>'

RESULTS

Worked Example #1: Bounded Counter (0 to 9)

This example demonstrates a bounded counter implemented purely through connection topology. Pressing the keypad plus key increments the counter up to 9; pressing keypad minus decrements it down to 0. At each step, the active node reconfigures both the up and down connections, so the system's next behavior is entirely determined by its current topology. No numeric variable is tested or stored — the count is embodied in which node is currently active.

The full implementation script is provided in **Appendix A**.

Worked Example #2: Ordered Grenade Selection System - Throwing a smoke grenade then throwing a high explosive grenade

This example demonstrates a bounded combinatorial controller — a system that manages the ordered selection and sequential execution of up to four discrete items, with slot-skipping and reset behavior, implemented entirely through connection redefinition.

The system models a combination lock with four dials (0–4), where each non-zero value can appear at most once. For $n=4$ items, this yields 209 valid permutations, each reachable within 4 keystrokes. The access depth scales linearly with n , not exponentially.

The system is initialized, grenade types are assigned to slots through sequential key presses, and throwing proceeds by rotating through occupied slots automatically, skipping empty ones. With each throw, the system emits the grenade type to the console.

This example illustrates that the full combinatorial state space — selection order, slot assignment, skip logic, and reset — is encoded exclusively in the current connection topology, with no numeric counters, no conditional branches written as instructions, and no explicit state variables anywhere in the script layer.

The full implementation script is provided in **Appendix B**.

Scope and Environmental Constraints:

A potential falsification challenge is that alias command names — such as `v0` or `gbutrot` — constitute symbols, and therefore the system is not genuinely symbol-free. This challenge, while formally valid in a general computational model, does not apply within the scope of this framework. Logic Geometry is defined and confined to the Quakeworld scripting environment, where alias names are fixed infrastructure provided by the host engine — not data structures the logic creates, reads, or evaluates. The computation does not branch based on a name's value; no alias name is ever passed as an argument, tested for equality, or used as a measurable quantity within the script. In this sense, alias names occupy the same role as physical wire connections in analog hardware: they are the medium through which topology is expressed, not symbols that the system processes. The claim of symbol-elimination therefore holds at the level of abstraction at which Logic Geometry operates — the scripting layer — and is explicitly bounded to that layer. Whether the underlying engine implementation encodes these connections as string-keyed hash entries in RAM is a fact about the host environment, not about the computational model being demonstrated, just as the voltage levels inside a transistor do not constitute "numerical computation" in the logical gate abstraction built atop them.

FORMAL SYSTEM

The Graph

At any frozen moment in time, the system looks like a directed graph. Each node is a named command. Each edge is a connection — it points from one node to the next node it will call. The graph is not fixed. It rewires itself as it is traversed. A node you visit may, as part of being visited, redirect where other nodes point — including nodes you are about to visit in the same traversal.

This means the graph you are walking changes underneath you as you walk it.

The Traversal

The operator initiates a traversal by pressing a key. The system follows one path through the current graph from that entry point to a dead end. That single traversal is one step of computation. The traversal is always finite — there is always a dead end, because the node set is finite and loops require the operator to re-initiate.

The Memory

There is no separate memory structure. The graph itself is the memory. Whatever the system remembers, it remembers by having reconfigured its own connections. If the system needs to "remember" that a counter is currently at 5, it does so by having the relevant nodes pointing in the specific pattern that produces 5-like behavior on the next traversal. The information is not stored — it is embodied in the shape.

The Logic

There are no if-then instructions. Logic emerges from the fact that a previous traversal already rerouted the connections. When the operator presses a key and the system behaves one way rather than another, that difference is entirely accounted for by the current shape of the graph. The graph shape is not a representation of the logic — it is the logic, physically instantiated as connections.

The Output

The system has one outward channel: text emitted to the console during a traversal. This is not a return value. It is more like a side effect that the operator observes. The system does not hand anything back — it speaks while it runs.

ACKNOWLEDGEMENTS

This research owes its foundational impetus to the scripting capabilities embedded within the Quake engine, designed by John Carmack, which enabled the systematic exploration of measurement-free logic systems. [16, 17]

REFERENCES

1. Păun, G. (2000). Computing with membranes. *Journal of Computer and System Sciences*

2. Păun, G. (2002). Membrane computing: An introduction. *Springer-Verlag*, Berlin.
3. Păun, G., Rozenberg, G., & Salomaa, A. (Eds.). (2010). *The Oxford Handbook of Membrane Computing*. Oxford University Press.
4. Soloveichik, D., Cook, M., Winfree, E., & Bruck, J. (2008). Computation with finite stochastic chemical reaction networks. *Natural Computing*
5. Cardelli, L., & Csikász-Nagy, A. (2012). The cell cycle switch computes approximate majority. *Scientific Reports*
6. Chen, Y.-J., Dalchau, N., Srinivas, N., Phillips, A., Cardelli, L., Soloveichik, D., & Seelig, G. (2013). Programmable chemical controllers made from DNA. *Nature Nanotechnology*
7. Petri, C. A. (1962). *Kommunikation mit Automaten* [Communication with automata]. PhD thesis, Universität Bonn, Germany
8. Murata, T. (1989). Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*
9. Reisig, W., & Rozenberg, G. (Eds.). (1998). *Lectures on Petri Nets I: Basic Models*. Springer-Verlag, Berlin. Lecture Notes in Computer Science, Vol. 1491.
10. Ehrig, H., Pfender, M., & Schneider, H. J. (1973). Graph-grammars: An algebraic approach. In *14th Annual Symposium on Switching and Automata Theory*. IEEE.
11. Plump, D. (1999). Term graph rewriting. In H. Ehrig, G. Engels, H.-J. Kreowski, & G. Rozenberg (Eds.), *Handbook of Graph Grammars and Computing by Graph Transformation* (Vol. 2,). World Scientific.
12. Kreowski, H.-J., & Kuske, S. (1999). Graph transformation units with interleaving semantics. *Formal Aspects of Computing*,
13. Church, A. (1936). An unsolvable problem of elementary number theory. *American Journal of Mathematics*
14. Barendregt, H. P. (1984). *The Lambda Calculus: Its Syntax and Semantics* (2nd ed.). North-Holland.
15. Cardelli, L. (2004). Type systems. In J. van Leeuwen, G. Rozenberg, & A. Salomaa (Eds.), *Handbook of Theoretical Computer Science, Volume B: Formal Models and Semantics*. Elsevier.
16. Abrash, M. (1997). Michael Abrash's graphics programming black book, special edition. Coriolis Group Books.
17. Carmack, J. (1996). Quake engine source code and design notes. id Software. (Released under GPL in 1999; archived at <https://github.com/id-Software/Quake>)

APPENDIX A — Worked Example #1: Bounded Counter Script

Press KP_PLUS to increment by one up to 9; press KP_MINUS to decrement down by one to 0.

```
bind KP_PLUS up
bind KP_MINUS dn
alias v0 "echo 0;alias up v1; alias dn v0"
alias v1 "echo 1; alias up v2; alias dn v0"
alias v2 "echo 2;alias up v3; alias dn v1"
alias v3 "echo 3;alias up v4; alias dn v2"
```



```
alias v4 "echo 4;alias up v5; alias dn v3"
alias v5 "echo 5;alias up v6; alias dn v4"
alias v6 "echo 6;alias up v7; alias dn v5"
alias v7 "echo 7;alias up v8; alias dn v6"
alias v8 "echo 8;alias up v9; alias dn v7"
alias v9 "echo 9;alias up v9; alias dn v8"
```

APPENDIX B — Worked Example #2: Grenade Combination Lock Script

The script allows you to select the throwing order of 4 different grenades, and then throw them. The 0 in the combination lock represents an empty slot, which when throwing will be skipped over. The first slot button you push will take high explosive, the second slot button you push even if it is the same button as the high explosive grenade will put a flash bang in the slot, then teargas, then smoke. Pressing any slot a 5th time will reset the slots.

Initialization: A plus sign (+) in front of a command means that command is executed when the key is pressed down, the minus sign (-) in front of a command means that command is executed when the key is released.

The script begins with the system linked to `grnrt0`. This executes `grst1` and executes an “echo FULL RESET of Grenade Slots”. The command `grst1` links `grnslta` through `grnsltd` each incrementally to `grnrta1` through `grnrtd1` setting up the selection of grenades. The `gsrst` command, the `gbrst` command, and `skiprst` command are executed.

The `gsrst` command links the the grenade slot place holders `+gs1` to `+gs4` to the null command with their release action of `-gs1` to `-gs4` to `gbutrot`. The `gbutrot` command causes the rotation of the grenades to be thrown.

The `gbrst` command links `gbutrot` to the first grenade throw slot linker, `gbutrot1` and `+gtoss` is set to `+gtall` and `-gtoss` is set to `-gtall` to default throw any grenade you have.

The command `skiprst` links `skip1` through `skip4` to `skip1rst` through `skip4rst`

The commands `gbutrot1` through `gbutrot4` increment the current `+gtoss` command to `+gs1` through `+gs4` slot placeholders and `-gtoss` command to `-gs1` through `-gs4` slot placeholders, with `gbutrot` linked to the next `gbutrot2` through `gbutrot4` with a `skip1` through `skip4` to jump over empty slots as the user is throwing or halt rotation for a selected grenade throw.

The commands `skip1rst` through `skip4rst` work with `gbutrot1` through `gbutrot4` incrementally with the `skip1` through `skip4` commands to rotate grenades for throwing. The commands `skip1rst` through `skip4rst` incrementally set the `+gs1` through `+gs4` slot placeholders for grenades to `+gtoss` and `-gtoss`, then sets `gbutrot` to the next `gbutrot3` or `gbutrot4` or `skip4rst`, and then executes `skip2` through `skip4` to skip empty slots or halt at that slot for throwing. The command `skip4rst` resets everything to throw any grenade you have.

Grenade Selection:

The commands `grnslta1` through `grnsltd4` incrementally execute `grst2` through `grst5` incrementing the commands `grnslta` through `grnsltd` connected to operator keys to only use each type of grenade once. The commands `grnslta1` through `grnsltd4` also link the `+gs1` and `-gs1` through `+gs4` and `-gs4` commands to which specific type of grenade to throw; `+gthe` and

-gtthe (high explosive), +gtfb and -gtfb (flash bang), +gttg and -gttg (tear gas), or +gtsmk and -gtsmk (smoke grenade). The commands grnslt**a1** through grnslt**d4** also link the corresponding skip**n** to execute the command null, which stops the execution of the loop.

First the V key is pressed to put a high explosive in slot 3 through executing grnslt**c** which executes grnslt**c1**. This links +gtthe and -gtthe to +gs**3** and -gs**3**, and skip**3** to null so the script stops on the third slot to throw the grenade. The command echo Grenade Slot 3 high explosive grenade is executed.

The X key is pressed once repeating the execution for high explosive, to put +gtfb and -gtfb in +gs**1** and -gs**1** and set skip**1** to null so the script stops on the first slot to throw a grenade. The X key is pressed again to put +gttg and -gttg in +gs**1** and -gs**1**. Once more, press the X key to put +gtsmk and -gtsmk in +gs**1** and -gs**1**.

The system is now set to throw a smoke grenade first, skip the second slot, throw the high explosive second, skip the fourth and last slot, then go to throwing any grenade you might have.

Throwing Mechanics:

The command gbutrot executes gbutrot**1** through gbutrot**4**, which incrementally link to the grenade slots +gs**1** through +gs**4** and execute a corresponding skip**1** through skip**4**.

The Mouse2 key is pressed executing +gtoss which executes +gs**1** which executes +gtsmk. The Mouse2 key is released, -gs**1** executes gbutrot which executes gbutrot**2** to set up the next Mouse2 press. The commands +gs**2** and -gs**2** are skipped to the third slot executing skip**2** which executes skip**2**rst. The Mouse2 key is pressed executing +gtoss which executes +gs**3** which executes +gtthe. When the Mouse2 key is released, -gs**3** executes gbutrot which executes gbutrot**4**. The command +gs**4** and -gs**4** are skipped by executing skip**4** which executes skip**4**rst that sets the Mouse2 grenade throwing to throw any grenade available.

With each toss it executes echo and the grenade thrown.

Start of grenade-combination_lock.cfg

```
-----
//bind keys to commands for user input
bind x grnsla //bind the X key to grenade slot 1
bind c grnsltb //bind the C key to grenade slot 2
bind v grnsltc //bind the V key to grenade slot 3
bind b grnsltd //bind the B key to grenade slot 4
bind n grnrt0 //bind the N key to reset the grenade slots
bind MOUSE2 +gtoss //bind mouse2 to the action of throwing the grenades

// Grenade Throwing System
alias null ""
alias gsrst "alias +gs1 null;alias -gs1 gbutrot;alias +gs2 null;alias -gs2 gbutrot;alias +gs3 null;alias -gs3 gbutrot;alias +gs4
null;alias -gs4 gbutrot"
alias gbrst "alias gbutrot gbutrot1;alias +gtoss +gtall;alias -gtoss -gtall"
alias skiprst "alias skip1 skip1rst;alias skip2 skip2rst;alias skip3 skip3rst;alias skip4 skip4rst"

alias skip1rst "alias +gtoss +gs2;alias -gtoss -gs2;alias gbutrot gbutrot3;skip2"
alias skip2rst "alias +gtoss +gs3;alias -gtoss -gs3;alias gbutrot gbutrot4;skip3"
alias skip3rst "alias +gtoss +gs4;alias -gtoss -gs4;alias gbutrot skip4rst;skip4"
alias skip4rst "alias +gtoss +gtall;alias -gtoss -gtall;alias gbutrot null"
```

```
alias grst1 "alias grnslta grnrta1;alias grnsltb grnrta1;alias grnsltc grnrta1;alias grnsldt grnrtd1;gsrst;gbrst;skiprst"
alias grst2 "alias grnslta grnrta2;alias grnsltb grnrta2;alias grnsltc grnrta2;alias grnsldt grnrtd2"
alias grst3 "alias grnslta grnrta3;alias grnsltb grnrta3;alias grnsltc grnrta3;alias grnsldt grnrtd3"
alias grst4 "alias grnslta grnrta4;alias grnsltb grnrta4;alias grnsltc grnrta4;alias grnsldt grnrtd4"
alias grst5 "alias grnslta grnrta0;alias grnsltb grnrta0;alias grnsltc grnrta0;alias grnsldt grnrtd0"
```

```
alias grnrtd0 "grst1;echo FULL RESET of Grenade Slots"
alias grnrta1 "grst2;alias +gs1 +gthe;alias -gs1 -gthe;alias skip1 null;echo Grenade Slot 1 high explosive grenade"
alias grnrta2 "grst3;alias +gs1 +gtfb;alias -gs1 -gtfb;alias skip1 null;echo Grenade Slot 1 flashbang grenade"
alias grnrta3 "grst4;alias +gs1 +gttg;alias -gs1 -gttg;alias skip1 null;echo Grenade Slot 1 teargas grenade"
alias grnrta4 "grst5;alias +gs1 +gtsmk;alias -gs1 -gtsmk;alias skip1 null;echo Grenade Slot 1 smoke grenade"
alias grnrta1 "grst2;alias +gs2 +gthe;alias -gs2 -gthe;alias skip2 null;echo Grenade Slot 2 high explosive grenade"
alias grnrta2 "grst3;alias +gs2 +gtfb;alias -gs2 -gtfb;alias skip2 null;echo Grenade Slot 2 flashbang grenade"
alias grnrta3 "grst4;alias +gs2 +gttg;alias -gs2 -gttg;alias skip2 null;echo Grenade Slot 2 teargas grenade"
alias grnrta4 "grst5;alias +gs2 +gtsmk;alias -gs2 -gtsmk;alias skip2 null;echo Grenade Slot 2 smoke grenade"
alias grnrta1 "grst2;alias +gs3 +gthe;alias -gs3 -gthe;alias skip3 null;echo Grenade Slot 3 high explosive grenade"
alias grnrta2 "grst3;alias +gs3 +gtfb;alias -gs3 -gtfb;alias skip3 null;echo Grenade Slot 3 flashbang grenade"
alias grnrta3 "grst4;alias +gs3 +gttg;alias -gs3 -gttg;alias skip3 null;echo Grenade Slot 3 teargas grenade"
alias grnrta4 "grst5;alias +gs3 +gtsmk;alias -gs3 -gtsmk;alias skip3 null;echo Grenade Slot 3 smoke grenade"
alias grnrtd1 "grst2;alias +gs4 +gthe;alias -gs4 -gthe;alias skip4 null;echo Grenade Slot 4 high explosive grenade"
alias grnrtd2 "grst3;alias +gs4 +gtfb;alias -gs4 -gtfb;alias skip4 null;echo Grenade Slot 4 flashbang grenade"
alias grnrtd3 "grst4;alias +gs4 +gttg;alias -gs4 -gttg;alias skip4 null;echo Grenade Slot 4 teargas grenade"
alias grnrtd4 "grst5;alias +gs4 +gtsmk;alias -gs4 -gtsmk;alias skip4 null;echo Grenade Slot 4 smoke grenade"
```

```
alias gbutrot1 "alias +gtoss +gs1;alias -gtoss -gs1;alias gbutrot gbutrot2;skip1"
alias gbutrot2 "alias +gtoss +gs2;alias -gtoss -gs2;alias gbutrot gbutrot3;skip2"
alias gbutrot3 "alias +gtoss +gs3;alias -gtoss -gs3;alias gbutrot gbutrot4;skip3"
alias gbutrot4 "alias +gtoss +gs4;alias -gtoss -gs4;alias gbutrot skip4rst;skip4"
```

```
alias +gthe "echo throw high explosive grenade"
alias -gthe "gbutrot"
alias +gtfb "echo throw flashbang grenade"
alias -gtfb "gbutrot"
alias +gttg "echo throw teargas grenade"
alias -gttg "gbutrot"
alias +gtsmk "echo throw smoke grenade"
alias -gtsmk "gbutrot"
alias +gtall "echo throw any available grenade"
alias -gtall "null"
```

```
//initialize the system
```

```
grnrtd0
```

```
*End of grenade-combination_lock.cfg*
```